

HW2 - STAT 701

Wells Guan

Date: October 6, 2026

The appendix contains the Python source used for all simulations, estimation, diagnostics, and figures. The source files contain no inline comments.

Problem 1. Textbook exercises

Brockwell and Davis 3.1, 3.11, 4.8, 4.13. For 3.1, you are welcome to check for causality/invertibility with the assistance of a computer in the sense of using a computer program to compute, e.g., roots of a polynomial. You are not welcome to find some package that offers a function where you just dump in coefficients and get back the answer.

Ex. 3.1

Determine which of the following processes are causal and/or invertible:

- (a) $X_t + .2X_{t-1} - .48X_{t-2} = Z_t$;
- (b) $X_t + 1.9X_{t-1} + .88X_{t-2} = Z_t + .2Z_{t-1} + .7Z_{t-2}$;
- (c) $X_t + .6X_{t-2} = Z_t + 1.2Z_{t-1}$;
- (d) $X_t + 1.8X_{t-1} + .81X_{t-2} = Z_t$;
- (e) $X_t + 1.6X_{t-1} = Z_t - .4Z_{t-1} + .04Z_{t-2}$.

Sol.

- (a) Notice

$$\phi(z) = 1 + .2z - .48z^2 = (1 - .6z)(1 + .8z), \quad \theta(z) = 1.$$

The zeroes of ϕ are $5/3$ and $-5/4$, both of which have modulus greater than 1. Since θ has no zeroes, the process is both causal and invertible.

- (b) Notice

$$\phi(z) = 1 + 1.9z + .88z^2 = (1 + 1.1z)(1 + .8z), \quad \theta(z) = 1 + .2z + .7z^2.$$

The zeroes of ϕ are $-10/11$ and $-5/4$. Since $|-10/11| < 1$, the process is not causal. The two zeroes of θ are complex conjugates, and their common modulus is

$$\sqrt{\frac{1}{.7}} = \sqrt{\frac{10}{7}} > 1.$$

Hence the process is invertible.

- (c) Notice

$$\phi(z) = 1 + .6z^2, \quad \theta(z) = 1 + 1.2z.$$

The zeroes of ϕ are $\pm i\sqrt{5/3}$, whose modulus is greater than 1, so the process is causal. The zero of θ is $-5/6$, which lies inside the unit disk, so the process is not invertible.

(d) Notice

$$\phi(z) = 1 + 1.8z + .81z^2 = (1 + .9z)^2, \quad \theta(z) = 1.$$

The only zero of ϕ is $-10/9$, with multiplicity two, and it lies outside the unit disk. Thus the process is both causal and invertible.

(e) Notice

$$\phi(z) = 1 + 1.6z, \quad \theta(z) = 1 - .4z + .04z^2 = (1 - .2z)^2.$$

The zero of ϕ is $-5/8$, so the process is not causal. The only zero of θ is 5, with multiplicity two, so the process is invertible.

Ex. 3.11

Let $\{X_t\}$ be an ARMA process with $\phi(z) \neq 0$ for $|z| = 1$ and autocovariance function $\gamma(\cdot)$. Show that there exist constants $C > 0$ and $s \in (0, 1)$ such that

$$|\gamma(h)| \leq Cs^{|h|}, \quad h = 0, \pm 1, \dots,$$

and hence that

$$\sum_{h=-\infty}^{\infty} |\gamma(h)| < \infty.$$

Sol.

Let σ^2 be $\mathbb{E}Z_t^2$. On the unit circle, the spectral density of $\{X_t\}$ is

$$f(\lambda) = \frac{\sigma^2}{2\pi} \frac{\theta(e^{-i\lambda})\theta(e^{i\lambda})}{\phi(e^{-i\lambda})\phi(e^{i\lambda})}.$$

Consider the rational function

$$F(z) = \sigma^2 \frac{\theta(z)\theta(z^{-1})}{\phi(z)\phi(z^{-1})}.$$

Because ϕ has only finitely many zeroes and none lies on the unit circle, there are numbers $r < 1 < R$ such that F is analytic on the annulus

$$r < |z| < R,$$

and hence F has a Laurent expansion there,

$$F(z) = \sum_{h=-\infty}^{\infty} a_h z^h.$$

The Fourier inversion formula on $|z| = 1$ shows that, up to reversing the index according to the Fourier convention, these Laurent coefficients are the autocovariances. Since $\gamma(-h) = \gamma(h)$, we may write $|a_h| = |\gamma(h)|$.

Choose $r < r_0 < 1 < R_0 < R$, and set

$$M_+ = \max_{|z|=R_0} |F(z)|, \quad M_- = \max_{|z|=r_0} |F(z)|.$$

The Cauchy estimates for Laurent coefficients give, for $h \geq 0$,

$$|a_h| \leq M_+ R_0^{-h}, \quad |a_{-h}| \leq M_- r_0^h.$$

Let

$$s = \max\{R_0^{-1}, r_0\} < 1, \quad C = \max\{M_+, M_-\}.$$

It follows that

$$|\gamma(h)| \leq C s^{|h|}, \quad h \in \mathbb{Z}.$$

Finally,

$$\sum_{h=-\infty}^{\infty} |\gamma(h)| \leq C \sum_{h=-\infty}^{\infty} s^{|h|} = C \left(1 + 2 \sum_{h=1}^{\infty} s^h \right) = C \frac{1+s}{1-s} < \infty.$$

Ex. 4.8

Let $\{X_t\}$ and $\{Y_t\}$ be stationary zero-mean processes with spectral densities f_X and f_Y . If

$$f_X(\lambda) \leq f_Y(\lambda) \quad \text{for all } \lambda \in [-\pi, \pi],$$

show that

- (a) $\Gamma_{n,Y} - \Gamma_{n,X}$ is a non-negative definite matrix, where $\Gamma_{n,Y}$ and $\Gamma_{n,X}$ are the covariance matrices of $\mathbf{Y} = (Y_1, \dots, Y_n)'$ and $\mathbf{X} = (X_1, \dots, X_n)'$, respectively; and
- (b) $\text{Var}(\mathbf{b}'\mathbf{X}) \leq \text{Var}(\mathbf{b}'\mathbf{Y})$ for all $\mathbf{b} = (b_1, \dots, b_n)' \in \mathbb{R}^n$.

Sol.

For any $\mathbf{b} = (b_1, \dots, b_n)' \in \mathbb{R}^n$, stationarity and the spectral representation of the autocovariance function give

$$\begin{aligned} \mathbf{b}'\Gamma_{n,X}\mathbf{b} &= \sum_{j=1}^n \sum_{k=1}^n b_j b_k \gamma_X(j-k) \\ &= \int_{-\pi}^{\pi} \left| \sum_{j=1}^n b_j e^{ij\lambda} \right|^2 f_X(\lambda) d\lambda. \end{aligned}$$

The same identity holds for Y . Therefore

$$\begin{aligned} \mathbf{b}'(\Gamma_{n,Y} - \Gamma_{n,X})\mathbf{b} &= \int_{-\pi}^{\pi} \left| \sum_{j=1}^n b_j e^{ij\lambda} \right|^2 (f_Y(\lambda) - f_X(\lambda)) d\lambda \\ &\geq 0, \end{aligned}$$

because both factors in the integrand are non-negative. This holds for every $\mathbf{b} \in \mathbb{R}^n$, so $\Gamma_{n,Y} - \Gamma_{n,X}$ is non-negative definite, proving (a).

Since the processes have mean zero,

$$\text{Var}(\mathbf{b}'\mathbf{X}) = \mathbf{b}'\Gamma_{n,X}\mathbf{b}, \quad \text{Var}(\mathbf{b}'\mathbf{Y}) = \mathbf{b}'\Gamma_{n,Y}\mathbf{b}.$$

The inequality just established thus immediately yields

$$\text{Var}(\mathbf{b}'\mathbf{X}) \leq \text{Var}(\mathbf{b}'\mathbf{Y}),$$

which proves (b).

Ex. 4.13

If $\{X_t\}$ and $\{Y_t\}$ are stationary processes satisfying

$$X_t - \alpha X_{t-1} = W_t, \quad \{W_t\} \sim \text{WN}(0, \sigma^2),$$

and

$$Y_t - \alpha Y_{t-1} = X_t + Z_t, \quad \{Z_t\} \sim \text{WN}(0, \sigma^2),$$

where $|\alpha| < 1$ and $\{W_t\}$ and $\{Z_t\}$ are uncorrelated, find the spectral density of $\{Y_t\}$.

Sol.

Using the backshift operator B , the first equation gives

$$X_t = \frac{1}{1 - \alpha B} W_t.$$

Substituting this into the second equation yields

$$\begin{aligned} Y_t &= \frac{1}{1 - \alpha B} (X_t + Z_t) \\ &= \frac{1}{(1 - \alpha B)^2} W_t + \frac{1}{1 - \alpha B} Z_t. \end{aligned}$$

Because $|\alpha| < 1$, both filters are stable. Moreover, $\{W_t\}$ and $\{Z_t\}$ are uncorrelated, so the two filtered processes above have zero cross-spectrum. Their spectral densities therefore add, giving

$$\begin{aligned} f_Y(\lambda) &= \frac{\sigma^2}{2\pi} \left(\frac{1}{|1 - \alpha e^{-i\lambda}|^4} + \frac{1}{|1 - \alpha e^{-i\lambda}|^2} \right) \\ &= \frac{\sigma^2}{2\pi} \frac{2 + \alpha^2 - 2\alpha \cos \lambda}{(1 + \alpha^2 - 2\alpha \cos \lambda)^2}, \quad -\pi \leq \lambda \leq \pi, \end{aligned}$$

where we used

$$|1 - \alpha e^{-i\lambda}|^2 = 1 + \alpha^2 - 2\alpha \cos \lambda.$$

Problem 2.

An ARIMA(p, d, q) (where the I is for *integrated*) process is a time series $\{X_t\}$ where $(I - B)^d X_t$ is an ARMA(p, q) process. A SARIMA(s, p, d, q), where the S is for *seasonal*, process is a time series where $(I - B^s)X_t$ is an ARIMA(p, d, q) process. Let's explore simulating and fitting these extensions to data. For this question, you are welcome to find a package that fits (S)AR(I)MA models, but find one that lets you specify the degrees s, p, d, q manually! We'll talk about estimation in detail later in the semester, but I don't want to wait too long before asking you to start connecting the math to data.

1. Write a function in whatever programming language you like to simulate an ARMA(p, q) process in the state-space formulation. Importantly, please design this program to accept custom white noise generators. For a Gaussian white noise, generate a length $n = 1,000$ ARMA(p, q) process with, say, $p, q > 3$ and put it into the estimation function that you find to check your work.

Sol.

Consider

$$X_t = \sum_{j=1}^p \phi_j X_{t-j} + Z_t + \sum_{j=1}^q \theta_j Z_{t-j}.$$

If

$$\mathbf{s}_t = (X_t, \dots, X_{t-p+1}, Z_t, \dots, Z_{t-q+1})',$$

then this recursion has the state-space form

$$\mathbf{s}_t = A\mathbf{s}_{t-1} + \mathbf{g}Z_t,$$

where the first row of A is $(\phi_1, \dots, \phi_p, \theta_1, \dots, \theta_q)$, the remaining rows shift the stored observations and innovations by one coordinate, and $\mathbf{g}$ has ones in coordinates 1 and $p + 1$ and zeroes elsewhere. The function in the appendix implements this update and receives the innovation generator as an argument.

I simulated an ARMA(4, 4) process with

$$\boldsymbol{\phi} = (.8, -.3, .1, -.05), \quad \boldsymbol{\theta} = (-.6, .2, .1, .05), \quad Z_t \sim N(0, 1),$$

discarding a burn-in of 1,000 observations. I then fitted an ARMA(4, 4) model with the standard Gaussian maximum-likelihood routine in `statsmodels`. The estimates were

$$\hat{\boldsymbol{\phi}} = (.7469, -.0024, .0185, -.1247), \quad \hat{\boldsymbol{\theta}} = (-.5926, -.0418, .1559, .0776),$$

with $\hat{\sigma}^2 = 1.0179$. The leading AR and MA effects and the innovation variance are recovered well. Individual higher-order coefficients are less precisely identified from only 1,000 observations, which is common for a deliberately high-order ARMA model.

2. Now, using the same coefficients, generate a similar length dataset but with $Z_t \sim t_{2.1}$ (the t-distribution with $\nu = 2.1$ degrees of freedom). Run it through your estimation function again and comment on your results. How would you summarize the effect of very heavy tailed noise on generic estimation packages?

Sol.

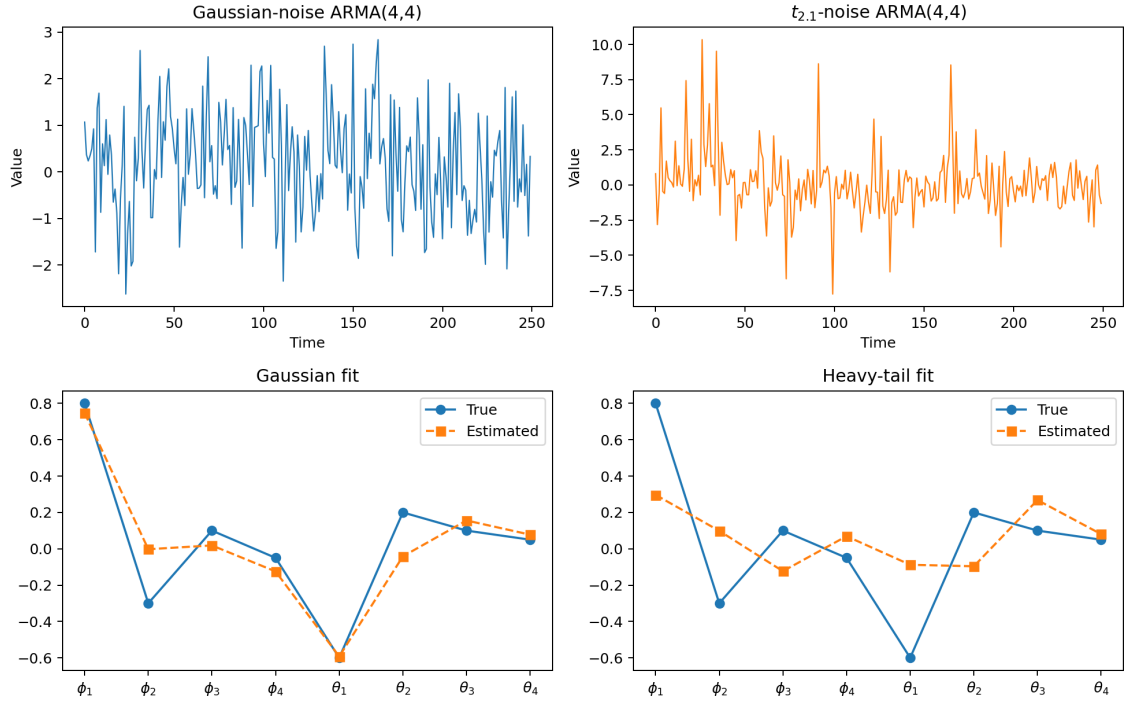


Figure 1: Gaussian and heavy-tailed ARMA simulations and fitted coefficients.

Using the same state transition and replacing only the innovation generator by $t_{2.1}$ gave

$$\hat{\phi} = (.2967, .0987, -.1233, .0692), \quad \hat{\theta} = (-.0880, -.0963, .2682, .0805),$$

and $\hat{\sigma}^2 = 104.2024$. For comparison, a $t_{2.1}$ variable has variance $2.1/(2.1 - 2) = 21$, but its fourth moment is infinite. Thus even a sample of length 1,000 can contain a few observations that dominate the Gaussian likelihood and make the sample variance far larger than its population value. Figure 1 shows the resulting isolated spikes and the substantial coefficient distortion. Generic Gaussian ARMA estimators can therefore be highly unstable under very heavy tails: the estimated innovation variance is especially sensitive, standard errors based on Gaussian moments are unreliable, and the dynamic coefficients may shift to accommodate a few extreme observations. Robust likelihoods or estimators designed for heavy tails are preferable in this setting.

3. Download `nao.csv`, the North Atlantic Oscillation time series from Canvas (and note there is a missing value, which you'll have to decide how to address). Using empirical ACFs and pACFs like I quickly demonstrated in class (both on your data and on differenced data and so on), do some exploratory analysis and propose and fit a $\text{SARIMA}(s, p, d, q)$ model. Please report your final model and provide justification for your choices. As a final validation, please simulate from your model and repeat the diagnostics on your simulated data. Discuss your results. In particular, try to propose the most important feature of this data that your SARIMA model is not capturing.

Sol.

The supplied file contains 17,410 daily observations from 1979 through August 2026. Although the filename is `nao.csv`, its value column is labelled `aao_index_cdas`; all calculations below use the

supplied column as given. The only missing observation is April 30, 2003. I replaced it by linear interpolation between the adjacent days.

The raw ACF is .9512 at lag 1, .8488 at lag 2, .4837 at lag 7, and .1132 at lag 30. It is only $-.0289$ at lag 365, so the annual component is much weaker than the short-range persistence. Nevertheless, because the observations are daily and the requested model is seasonal, I used $s = 365$. After applying $1 - B^{365}$, the pACF has its main values at lags 1 through 5 and is close to zero thereafter, while the ACF decays gradually. This led to the parsimonious choice SARIMA(365, 4, 0, 2):

$$(1 - .6642B - .3518B^2 + .2324B^3 - .0837B^4)(1 - B^{365})X_t = (1 + 1.1556B + .3247B^2)Z_t,$$

with estimated innovation variance $\hat{\sigma}^2 = .1956$.

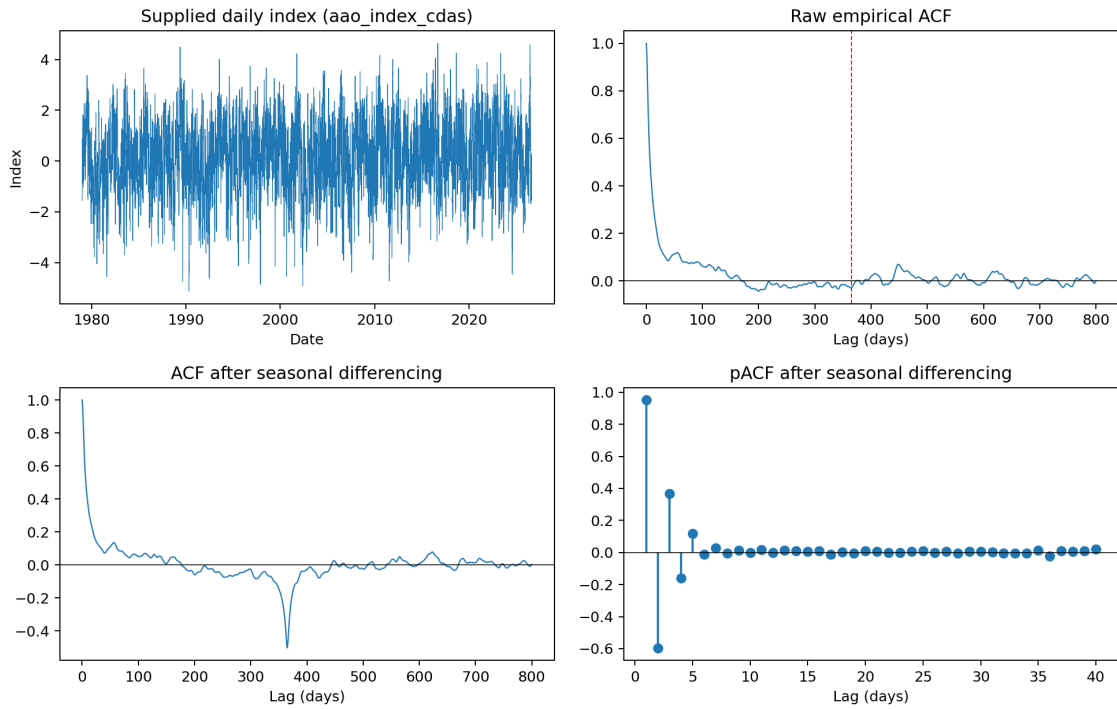


Figure 2: The supplied index and empirical ACF/pACF diagnostics.

I simulated the fitted ARMA equation for the seasonally differenced series and then reconstructed a series recursively from its first 365 values. The short-lag ACF of the simulated differences is close to the observed one, and the fitted residual correlations are all below .027 in magnitude. The model does not reproduce the slower positive ACF extending across several dozen days, and several residual correlations exceed the approximate 95% bounds. The main omitted feature is therefore a slowly changing dependence structure: a single constant-coefficient linear model cannot simultaneously capture the very strong short-run persistence, the broader low-frequency variation, and changes in volatility over the roughly 47-year span. The weak empirical correlation at lag 365 also suggests that annual seasonal differencing is a fairly rigid description of these data.

4* an ARIMA($p, 1, q$) process is effectively an ARMA(p, q) process with a “unit root”, meaning $\phi(z) = 0$ for some $|z| = 1$. Interpret the implication of this addition to the model in the spectral domain. How

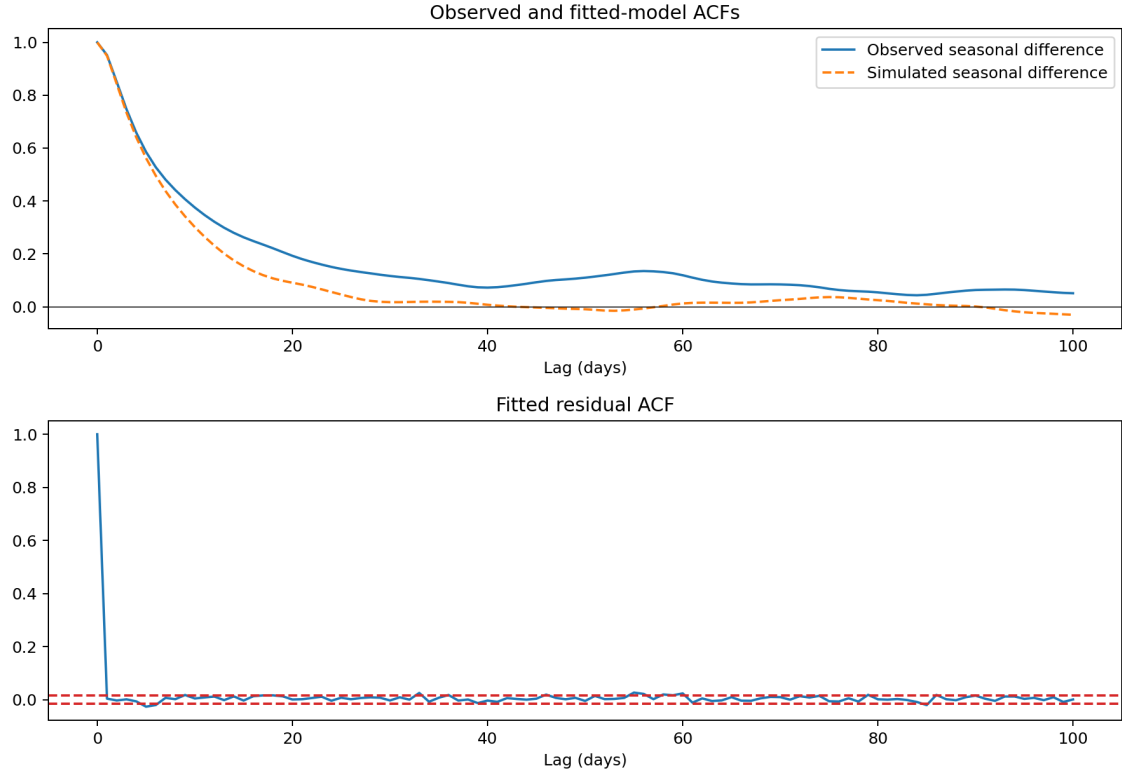


Figure 3: Validation using a simulation from the fitted seasonal model.

is the ARMA spectral density impacted? What are the downsides of such a design? Try and propose an extension of ARMA models in the same spirit, but that addresses your identified downside.

Sol.

For

$$\phi(B)(1 - B)X_t = \theta(B)Z_t,$$

the formal spectral density is

$$f_X(\lambda) = \frac{\sigma^2}{2\pi} \frac{|\theta(e^{-i\lambda})|^2}{|\phi(e^{-i\lambda})|^2 |1 - e^{-i\lambda}|^2}.$$

Since $|1 - e^{-i\lambda}|^2 = 4 \sin^2(\lambda/2) \sim \lambda^2$ near zero, the unit root creates a second-order pole at frequency zero. The expression is not integrable there, reflecting the nonstationarity and unbounded variance of the level process. Differencing removes this pole, but an integer difference is rigid: it forces exactly this singularity and can overdifference data whose low-frequency persistence is strong but not a unit root. Forecast uncertainty for the level also grows without bound with the forecast horizon.

A natural extension is ARFIMA(p, d, q),

$$\phi(B)(1 - B)^d X_t = \theta(B)Z_t,$$

where d need not be an integer. Its spectrum contains the factor $|1 - e^{-i\lambda}|^{-2d}$. For $0 < d < 1/2$ the process is stationary but has a controlled singularity at zero and long-range dependence. Thus fractional differencing preserves low-frequency persistence without imposing the full nonstationary unit root.

Problem 3.

Consider a stationary time series $\{Y_t\}$ with mean zero and covariance sequence given by

$$\text{Cov}(Y_t, Y_{t-h}) = K(h) = \frac{2e^{-\theta/2}(-\theta \cos(\pi h) + e^{\theta/2}\theta + 2\pi h \sin(\pi h))}{\theta^2 + (2\pi h)^2}.$$

Let's fit an ARMA process to this and see what is going on.

1. Write code to simulate draws from this process of length $n = 1,000$ directly using the Cholesky factor of the covariance matrix. You are welcome to use built-in routines for the Cholesky factorization itself, but please otherwise write this code yourself.

Sol.

I used $\theta = 6$. Define the Toeplitz covariance matrix

$$\Gamma_{ij} = K(i - j), \quad 1 \leq i, j \leq 1000.$$

Since K is a covariance sequence, Γ is non-negative definite. Its Cholesky factorization is $\Gamma = LL'$.

If $\mathbf{z} \sim N(\mathbf{0}, I_{1000})$ and $\mathbf{y} = L\mathbf{z}$, then

$$\mathbb{E}\mathbf{y} = \mathbf{0}, \quad \text{Cov}(\mathbf{y}) = LIL' = \Gamma.$$

Thus the entries of $\mathbf{y}$ are one Gaussian draw with the required covariance. The appendix constructs Γ , computes L , and generates the draw directly in this way. A numerical diagonal adjustment of 10^{-12} was used only to avoid roundoff problems in the factorization.

2. Pick orders p and q (again, I suggest trying different combinations for max fun!) and fit an ARMA(p, q) model to that simulation. Plot the implied ARMA ACF against the true ACF and comment on the patterns. You are again welcome to use a package for both fitting and obtaining the ARMA ACF.

Sol.

I chose $p = q = 4$ as a compromise between flexibility near lag zero and avoiding an unnecessarily high-order fit. A Whittle fit to the simulated series gave

$$\begin{aligned} \hat{\phi} &= (.17383, -.26896, -.65793, .33627), \\ \hat{\theta} &= (.44727, .37068, .90769, .16742), \\ \hat{\sigma}^2 &= .23108. \end{aligned}$$

The fitted ACF reproduces the sharp fall from lag 0 to lag 1 and is already close to zero by lag 5. It also introduces small alternating correlations at intermediate lags. These oscillations are needed because a finite rational spectrum can only approximate the cusp of the true spectrum; the empirical ACF itself has additional fluctuations from the single finite sample.

- 3*. Show that this $K(h)$ corresponds to a spectral density given by $S(\omega) = e^{-\theta|\omega|}$. Plot this true spectral density, and in the same plot also add the ARMA model-implied SDF using your fitted parameters. Comment on the quality of the fit and any specific issues you see.

Sol.

Using frequency in cycles per observation, $\omega \in [-1/2, 1/2]$, the inverse Fourier transform of the

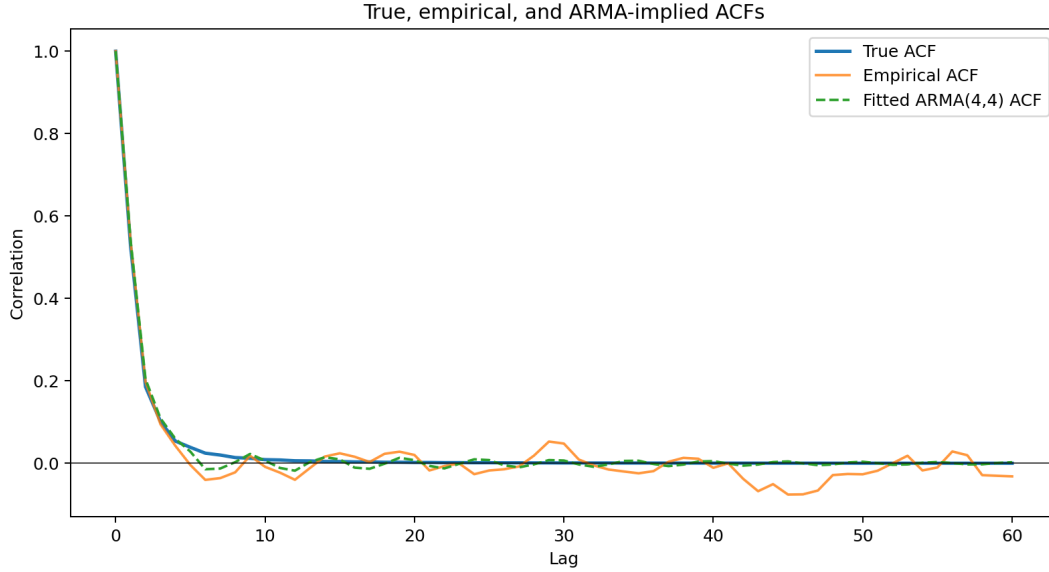


Figure 4: True, empirical, and fitted ARMA(4, 4) autocorrelations.

proposed density is

$$\begin{aligned} \int_{-1/2}^{1/2} e^{-\theta|\omega|} e^{2\pi i h \omega} d\omega &= 2 \int_0^{1/2} e^{-\theta\omega} \cos(2\pi h \omega) d\omega \\ &= \frac{2e^{-\theta/2}}{\theta^2 + (2\pi h)^2} \left[-\theta \cos(\pi h) + 2\pi h \sin(\pi h) + e^{\theta/2} \theta \right] \\ &= K(h). \end{aligned}$$

Hence $S(\omega) = e^{-\theta|\omega|}$ is the spectral density corresponding to K .

The time-series fit follows the overall decay away from frequency zero, but it rounds off the cusp at zero and creates two artificial notches and shoulders near frequencies $\pm.25$. Its root-mean-square log-spectral error over a dense grid is .11813. These local artifacts are the frequency-domain version of the small oscillations in the fitted ACF.

- 4*. Google around and choose an algorithm with a convenient package for fitting rational functions (you might look at the *Remez* or *AAA* algorithms, for example). Fit a rational function of the same order as your chosen ARMA process to directly to the true spectral density you verified in the last part, trying carefully to match the way that ARMA SDFs are parameterized. Compare your results with the fitted ARMA coefficients you obtained in 2 and discuss potential differences you observe.

Sol.

I used nonlinear rational least squares through `scipy.optimize.least_squares`. The fitted family was parameterized exactly as an ARMA(4, 4) spectrum,

$$S_{\text{ARMA}}(\omega) = \sigma^2 \frac{|1 + \sum_{j=1}^4 \theta_j e^{-2\pi i j \omega}|^2}{|1 - \sum_{j=1}^4 \phi_j e^{-2\pi i j \omega}|^2}.$$

Reflection coefficients constrained the AR and MA polynomials to the stable and invertible regions, and the criterion was least squares on the log spectrum over 2,001 equally spaced frequencies. The

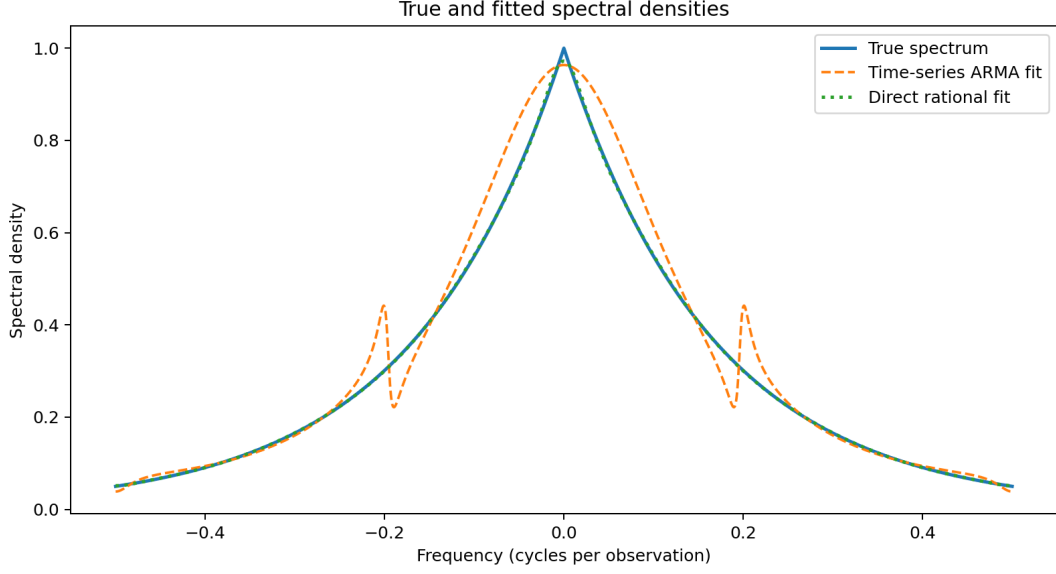


Figure 5: True spectrum, the fit to the simulated series, and the direct rational fit.

result was

$$\begin{aligned}\tilde{\boldsymbol{\phi}} &= (.28813, .68534, -.17504, -.02755), \\ \tilde{\boldsymbol{\theta}} &= (.31981, -.67572, -.18970, .02384), \\ \tilde{\sigma}^2 &= .22313.\end{aligned}$$

Its log-spectral root-mean-square error is .00416, far below .11813 for the time-series fit. The two coefficient vectors differ because the direct fit uses the exact target spectrum at every grid point, whereas the first fit sees only one random sample and minimizes a Whittle likelihood based on its noisy periodogram. Moreover, finite-order AR and MA factors can trade off while producing similar spectral shapes. As Figure 5 shows, the direct fit is nearly indistinguishable from the target except in a very small neighborhood of the nondifferentiable cusp at zero.

A Python code

The scripts read `code/nao.csv` relative to the HW2 directory and save all figures in `png/`. They use NumPy, pandas, SciPy, Matplotlib, and statsmodels.

A.1 Problem 2

```
1 from pathlib import Path
2 import numpy as np
3 import pandas as pd
4 import matplotlib
5 matplotlib.use("Agg")
6 import matplotlib.pyplot as plt
7 from scipy.linalg import toeplitz
8 from scipy.optimize import minimize
9 from statsmodels.tsa.arima.model import ARIMA
10
11
12 ROOT = Path(__file__).resolve().parents[1]
13 PNG = ROOT / "png"
14 PNG.mkdir(exist_ok=True)
15
16
17 def reflection_to_ar(values):
18     kappa = np.tanh(np.asarray(values, dtype=float))
19     if len(kappa) == 0:
20         return np.array([])
21     coefficients = np.array([kappa[0]])
22     for m in range(1, len(kappa)):
23         previous = coefficients.copy()
24         coefficients = np.empty(m + 1)
25         coefficients[:m] = previous - kappa[m] * previous[::-1]
26         coefficients[m] = kappa[m]
27     return coefficients
28
29
30 def simulate_arma_state(ar, ma, n, noise_generator, seed=701, burn=1000):
31     ar = np.asarray(ar, dtype=float)
32     ma = np.asarray(ma, dtype=float)
33     p = len(ar)
34     q = len(ma)
35     dimension = p + q
36     transition = np.zeros((dimension, dimension))
37     transition[0, :p] = ar
38     transition[0, p:] = ma
39     if p > 1:
40         transition[1:p, :p - 1] = np.eye(p - 1)
```

```

41     if q > 1:
42         transition[p + 1:, p:p + q - 1] = np.eye(q - 1)
43     loading = np.zeros(dimension)
44     loading[0] = 1.0
45     if q > 0:
46         loading[p] = 1.0
47     rng = np.random.default_rng(seed)
48     innovations = noise_generator(rng, n + burn)
49     state = np.zeros(dimension)
50     output = np.empty(n + burn)
51     for t, innovation in enumerate(innovations):
52         state = transition @ state + loading * innovation
53         output[t] = state[0]
54     return output[burn:]
55
56
57 def spectral_ratio(lam, ar, ma):
58     ar_power = np.arange(1, len(ar) + 1)
59     ma_power = np.arange(1, len(ma) + 1)
60     ar_poly = np.ones(len(lam), dtype=complex)
61     ma_poly = np.ones(len(lam), dtype=complex)
62     if len(ar):
63         ar_poly -= np.exp(-1j * np.outer(lam, ar_power)) @ ar
64     if len(ma):
65         ma_poly += np.exp(-1j * np.outer(lam, ma_power)) @ ma
66     return np.abs(ma_poly) ** 2 / np.abs(ar_poly) ** 2
67
68
69 def fit_whittle(x, p, q, seed=0, starts=4):
70     centered = np.asarray(x, dtype=float) - np.mean(x)
71     n = len(centered)
72     transform = np.fft.rfft(centered)
73     lam = 2 * np.pi * np.fft.rfftfreq(n)[1:]
74     periodogram = np.abs(transform[1:]) ** 2 / (2 * np.pi * n)
75
76     def decode(parameters):
77         ar = reflection_to_ar(parameters[:p])
78         ma = -reflection_to_ar(parameters[p:p + q])
79         return ar, ma
80
81     def objective(parameters):
82         ar, ma = decode(parameters)
83         ratio = spectral_ratio(lam, ar, ma)
84         scale = np.mean(periodogram / ratio)
85         model = np.maximum(scale * ratio, 1e-12)
86         return np.mean(np.log(model) + periodogram / model)
87

```

```

88     rng = np.random.default_rng(seed)
89     initial_values = [np.zeros(p + q)]
90     initial_values += [rng.normal(0, 0.2, p + q) for _ in range(starts - 1)]
91     fits = [minimize(objective, value, method="L-BFGS-B", options={"maxiter": 700})
92              for value in initial_values]
93     result = min(fits, key=lambda item: item.fun)
94     ar, ma = decode(result.x)
95     ratio = spectral_ratio(lam, ar, ma)
96     scale = np.mean(periodogram / ratio)
97     return ar, ma, 2 * np.pi * scale, result.fun
98
99
100 def autocorrelation(x, nlags):
101     centered = np.asarray(x, dtype=float) - np.mean(x)
102     denominator = centered @ centered
103     return np.array([centered[:len(centered) - lag] @ centered[lag:] / denominator
104                      for lag in range(nlags + 1)])
105
106
107 def partial_autocorrelation(x, nlags):
108     centered = np.asarray(x, dtype=float) - np.mean(x)
109     gamma = np.array([centered[:len(centered) - lag] @ centered[lag:] / len(centered)
110                       for lag in range(nlags + 1)])
111     output = np.ones(nlags + 1)
112     for lag in range(1, nlags + 1):
113         output[lag] = np.linalg.solve(toeplitz(gamma[:lag]), gamma[1:lag + 1])[-1]
114     return output
115
116
117 def innovations(x, ar, ma):
118     x = np.asarray(x, dtype=float) - np.mean(x)
119     residuals = np.zeros(len(x))
120     for t in range(len(x)):
121         ar_part = sum(ar[j] * x[t - j - 1] for j in range(len(ar)) if t > j)
122         ma_part = sum(ma[j] * residuals[t - j - 1] for j in range(len(ma)) if t > j)
123         residuals[t] = x[t] - ar_part - ma_part
124     return residuals
125
126
127 def gaussian_noise(rng, n):
128     return rng.normal(size=n)
129
130
131 def heavy_noise(rng, n):
132     return rng.standard_t(2.1, size=n)
133
134

```

```

135 def fit_package(x, p, q):
136     result = ARIMA(
137         x,
138         order=(p, 0, q),
139         trend="n",
140         enforce_stationarity=True,
141         enforce_invertibility=True,
142     ).fit(method_kwargs={"maxiter": 1000})
143     return result.arparams, result.maparams, result.params[-1]
144
145
146 true_ar = np.array([0.8, -0.3, 0.1, -0.05])
147 true_ma = np.array([-0.6, 0.2, 0.1, 0.05])
148 gaussian = simulate_arma_state(true_ar, true_ma, 1000, gaussian_noise, 801)
149 heavy = simulate_arma_state(true_ar, true_ma, 1000, heavy_noise, 802)
150 gaussian_ar, gaussian_ma, gaussian_variance = fit_package(gaussian, 4, 4)
151 heavy_ar, heavy_ma, heavy_variance = fit_package(heavy, 4, 4)
152
153 fig, axes = plt.subplots(2, 2, figsize=(11, 7))
154 axes[0, 0].plot(gaussian[:250], linewidth=0.9)
155 axes[0, 0].set_title("Gaussian-noise ARMA(4,4)")
156 axes[0, 0].set_xlabel("Time")
157 axes[0, 0].set_ylabel("Value")
158 axes[0, 1].plot(heavy[:250], linewidth=0.9, color="tab:orange")
159 axes[0, 1].set_title(r"$t_{2.1}$-noise ARMA(4,4)")
160 axes[0, 1].set_xlabel("Time")
161 axes[0, 1].set_ylabel("Value")
162 locations = np.arange(8)
163 truth = np.r_[true_ar, true_ma]
164 axes[1, 0].plot(locations, truth, "o-", label="True")
165 axes[1, 0].plot(locations, np.r_[gaussian_ar, gaussian_ma], "s--", label="Estimated")
166 axes[1, 0].set_xticks(locations, [r"$\phi_1$", r"$\phi_2$", r"$\phi_3$", r"$\phi_4$",
167                                   r"$\theta_1$", r"$\theta_2$", r"$\theta_3$", r"$\theta_4$"])
168 axes[1, 0].set_title("Gaussian fit")
169 axes[1, 0].legend()
170 axes[1, 1].plot(locations, truth, "o-", label="True")
171 axes[1, 1].plot(locations, np.r_[heavy_ar, heavy_ma], "s--", label="Estimated")
172 axes[1, 1].set_xticks(locations, [r"$\phi_1$", r"$\phi_2$", r"$\phi_3$", r"$\phi_4$",
173                                   r"$\theta_1$", r"$\theta_2$", r"$\theta_3$", r"$\theta_4$"])
174 axes[1, 1].set_title("Heavy-tail fit")
175 axes[1, 1].legend()
176 fig.tight_layout()
177 fig.savefig(PNG / "problem2_arma_fits.png", dpi=180)
178 plt.close(fig)
179
180 data = pd.read_csv(ROOT / "code" / "nao.csv")
181 value_column = [column for column in data.columns if column not in {"year", "month", "day"}][0]

```

```

182 dates = pd.to_datetime(data[["year", "month", "day"]]).to_numpy()
183 values = data[value_column].to_numpy(dtype=float)
184 missing_index = np.flatnonzero(np.isnan(values))
185 valid = np.flatnonzero(~np.isnan(values))
186 values[missing_index] = np.interp(missing_index, valid, values[valid])
187 season = 365
188 seasonal_difference = values[season:] - values[:-season]
189 nao_ar, nao_ma, nao_variance, _ = fit_whittle(seasonal_difference, 4, 2, 705, 5)
190 nao_residuals = innovations(seasonal_difference, nao_ar, nao_ma)
191 simulated_difference = simulate_arma_state(
192     nao_ar,
193     nao_ma,
194     len(seasonal_difference),
195     lambda rng, n: rng.normal(scale=np.sqrt(nao_variance), size=n),
196     706,
197 )
198 simulated = np.empty(len(values))
199 simulated[:season] = values[:season]
200 for t in range(season, len(values)):
201     simulated[t] = simulated[t - season] + simulated_difference[t - season]
202
203 raw_acf = autocorrelation(values, 800)
204 diff_acf = autocorrelation(seasonal_difference, 800)
205 diff_pacf = partial_autocorrelation(seasonal_difference, 40)
206
207 fig, axes = plt.subplots(2, 2, figsize=(11, 7))
208 axes[0, 0].plot(dates, values, linewidth=0.45)
209 axes[0, 0].set_title(f"Supplied daily index ({value_column})")
210 axes[0, 0].set_xlabel("Date")
211 axes[0, 0].set_ylabel("Index")
212 axes[0, 1].plot(raw_acf, linewidth=0.9)
213 axes[0, 1].axhline(0, color="black", linewidth=0.6)
214 axes[0, 1].axvline(365, color="tab:red", linestyle="--", linewidth=0.8)
215 axes[0, 1].set_title("Raw empirical ACF")
216 axes[0, 1].set_xlabel("Lag (days)")
217 axes[1, 0].plot(diff_acf, linewidth=0.9)
218 axes[1, 0].axhline(0, color="black", linewidth=0.6)
219 axes[1, 0].set_title("ACF after seasonal differencing")
220 axes[1, 0].set_xlabel("Lag (days)")
221 axes[1, 1].stem(np.arange(1, 41), diff_pacf[1:], basefmt=" ")
222 axes[1, 1].axhline(0, color="black", linewidth=0.6)
223 axes[1, 1].set_title("pACF after seasonal differencing")
224 axes[1, 1].set_xlabel("Lag (days)")
225 fig.tight_layout()
226 fig.savefig(PNG / "problem2_nao_diagnostics.png", dpi=180)
227 plt.close(fig)
228

```

```

229 observed_acf = autocorrelation(seasonal_difference, 100)
230 simulated_acf = autocorrelation(simulated_difference, 100)
231 residual_acf = autocorrelation(nao_residuals[50:], 100)
232
233 fig, axes = plt.subplots(2, 1, figsize=(10, 7))
234 axes[0].plot(observed_acf, label="Observed seasonal difference")
235 axes[0].plot(simulated_acf, label="Simulated seasonal difference", linestyle="--")
236 axes[0].axhline(0, color="black", linewidth=0.6)
237 axes[0].set_title("Observed and fitted-model ACFs")
238 axes[0].set_xlabel("Lag (days)")
239 axes[0].legend()
240 axes[1].plot(residual_acf)
241 axes[1].axhline(1.96 / np.sqrt(len(nao_residuals)), color="tab:red", linestyle="--")
242 axes[1].axhline(-1.96 / np.sqrt(len(nao_residuals)), color="tab:red", linestyle="--")
243 axes[1].set_title("Fitted residual ACF")
244 axes[1].set_xlabel("Lag (days)")
245 fig.tight_layout()
246 fig.savefig(PNG / "problem2_nao_validation.png", dpi=180)
247 plt.close(fig)
248
249 print("GAUSSIAN_AR", np.round(gaussian_ar, 4))
250 print("GAUSSIAN_MA", np.round(gaussian_ma, 4))
251 print("GAUSSIAN_VARIANCE", round(gaussian_variance, 4))
252 print("HEAVY_AR", np.round(heavy_ar, 4))
253 print("HEAVY_MA", np.round(heavy_ma, 4))
254 print("HEAVY_VARIANCE", round(heavy_variance, 4))
255 print("VALUE_COLUMN", value_column)
256 print("MISSING_ROWS", missing_index)
257 print("NAO_AR", np.round(nao_ar, 4))
258 print("NAO_MA", np.round(nao_ma, 4))
259 print("NAO_VARIANCE", round(nao_variance, 4))
260 print("RAW_ACF_1_365", np.round(raw_acf[[1, 2, 7, 30, 365]], 4))
261 print("DIFF_ACF_1_10", np.round(diff_acf[1:11], 4))
262 print("DIFF_PACF_1_10", np.round(diff_pacf[1:11], 4))
263 print("RESIDUAL_ACF_MAX", round(np.max(np.abs(residual_acf[1:])), 4))

```

A.2 Problem 3

```
1 from pathlib import Path
2 import numpy as np
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6 from scipy.linalg import toeplitz
7 from scipy.optimize import minimize, least_squares
8
9
10 ROOT = Path(__file__).resolve().parents[1]
11 PNG = ROOT / "png"
12 PNG.mkdir(exist_ok=True)
13
14
15 def covariance(h, theta):
16     h = np.asarray(h, dtype=float)
17     numerator = 2 * np.exp(-theta / 2) * (
18         -theta * np.cos(np.pi * h)
19         + np.exp(theta / 2) * theta
20         + 2 * np.pi * h * np.sin(np.pi * h)
21     )
22     return numerator / (theta ** 2 + (2 * np.pi * h) ** 2)
23
24
25 def reflection_to_ar(values):
26     kappa = np.tanh(np.asarray(values, dtype=float))
27     if len(kappa) == 0:
28         return np.array([])
29     coefficients = np.array([kappa[0]])
30     for m in range(1, len(kappa)):
31         previous = coefficients.copy()
32         coefficients = np.empty(m + 1)
33         coefficients[:m] = previous - kappa[m] * previous[::-1]
34         coefficients[m] = kappa[m]
35     return coefficients
36
37
38 def spectral_ratio(lam, ar, ma):
39     ar_power = np.arange(1, len(ar) + 1)
40     ma_power = np.arange(1, len(ma) + 1)
41     ar_poly = np.ones(len(lam), dtype=complex)
42     ma_poly = np.ones(len(lam), dtype=complex)
43     if len(ar):
44         ar_poly -= np.exp(-1j * np.outer(lam, ar_power)) @ ar
45     if len(ma):
```

```

46     ma_poly += np.exp(-1j * np.outer(lam, ma_power)) @ ma
47     return np.abs(ma_poly) ** 2 / np.abs(ar_poly) ** 2
48
49
50 def fit_whittle(x, p, q, seed=0, starts=8):
51     centered = np.asarray(x) - np.mean(x)
52     n = len(centered)
53     transform = np.fft.rfft(centered)
54     lam = 2 * np.pi * np.fft.rfftfreq(n)[1:]
55     periodogram = np.abs(transform[1:]) ** 2 / (2 * np.pi * n)
56
57     def decode(parameters):
58         return reflection_to_ar(parameters[:p]), -reflection_to_ar(parameters[p:p + q])
59
60     def objective(parameters):
61         ar, ma = decode(parameters)
62         ratio = spectral_ratio(lam, ar, ma)
63         scale = np.mean(periodogram / ratio)
64         model = np.maximum(scale * ratio, 1e-12)
65         return np.mean(np.log(model) + periodogram / model)
66
67     rng = np.random.default_rng(seed)
68     starts_list = [np.zeros(p + q)] + [rng.normal(0, 0.25, p + q) for _ in range(starts - 1)]
69     fits = [minimize(objective, start, method="L-BFGS-B", options={"maxiter": 900})
70             for start in starts_list]
71     result = min(fits, key=lambda item: item.fun)
72     ar, ma = decode(result.x)
73     ratio = spectral_ratio(lam, ar, ma)
74     variance = 2 * np.pi * np.mean(periodogram / ratio)
75     return ar, ma, variance
76
77
78 def empirical_acf(x, nlags):
79     centered = np.asarray(x) - np.mean(x)
80     denominator = centered @ centered
81     return np.array([centered[:len(centered) - lag] @ centered[lag:] / denominator
82                     for lag in range(nlags + 1)])
83
84
85 def spectral_acf(ar, ma, nlags, grid=32768):
86     lam = 2 * np.pi * np.arange(grid) / grid
87     spectrum = spectral_ratio(lam, ar, ma)
88     gamma = np.fft.ifft(spectrum).real[:nlags + 1]
89     return gamma / gamma[0]
90
91
92 def rational_fit(theta, p, q, seed=0, starts=8):

```

```

93 frequency = np.linspace(-0.5, 0.5, 2001)
94 target = np.exp(-theta * np.abs(frequency))
95 lam = 2 * np.pi * frequency
96
97 def decode(parameters):
98     ar = reflection_to_ar(parameters[:p])
99     ma = -reflection_to_ar(parameters[p:p + q])
100     scale = np.exp(parameters[-1])
101     return ar, ma, scale
102
103 def residual(parameters):
104     ar, ma, scale = decode(parameters)
105     model = np.maximum(scale * spectral_ratio(lam, ar, ma), 1e-12)
106     return np.log(model) - np.log(target)
107
108 rng = np.random.default_rng(seed)
109 initial_values = [np.zeros(p + q + 1)]
110 initial_values += [rng.normal(0, 0.2, p + q + 1) for _ in range(starts - 1)]
111 fits = [least_squares(residual, value, max_nfev=3000) for value in initial_values]
112 result = min(fits, key=lambda item: np.mean(item.fun ** 2))
113 return (*decode(result.x), np.sqrt(np.mean(result.fun ** 2)))
114
115
116 theta = 6.0
117 n = 1000
118 lags = np.arange(n)
119 matrix = toeplitz(covariance(lags, theta))
120 rng = np.random.default_rng(709)
121 sample = np.linalg.cholesky(matrix + 1e-12 * np.eye(n)) @ rng.normal(size=n)
122 p = 4
123 q = 4
124 time_ar, time_ma, time_variance = fit_whittle(sample, p, q, 710)
125 spectral_ar, spectral_ma, spectral_scale, spectral_rmse = rational_fit(theta, p, q, 711)
126
127 display_lags = 60
128 true_acf = covariance(np.arange(display_lags + 1), theta) / covariance(0, theta)
129 sample_acf = empirical_acf(sample, display_lags)
130 fitted_acf = spectral_acf(time_ar, time_ma, display_lags)
131
132 fig, ax = plt.subplots(figsize=(9, 5))
133 ax.plot(true_acf, label="True ACF", linewidth=2)
134 ax.plot(sample_acf, label="Empirical ACF", alpha=0.8)
135 ax.plot(fitted_acf, label="Fitted ARMA(4,4) ACF", linestyle="--")
136 ax.axhline(0, color="black", linewidth=0.6)
137 ax.set_xlabel("Lag")
138 ax.set_ylabel("Correlation")
139 ax.set_title("True, empirical, and ARMA-implied ACFs")

```

```

140 ax.legend()
141 fig.tight_layout()
142 fig.savefig(PNG / "problem3_acf_comparison.png", dpi=180)
143 plt.close(fig)
144
145 frequency = np.linspace(-0.5, 0.5, 2001)
146 lam = 2 * np.pi * frequency
147 true_spectrum = np.exp(-theta * np.abs(frequency))
148 time_spectrum = time_variance * spectral_ratio(lam, time_ar, time_ma)
149 direct_spectrum = spectral_scale * spectral_ratio(lam, spectral_ar, spectral_ma)
150
151 fig, ax = plt.subplots(figsize=(9, 5))
152 ax.plot(frequency, true_spectrum, label="True spectrum", linewidth=2)
153 ax.plot(frequency, time_spectrum, label="Time-series ARMA fit", linestyle="--")
154 ax.plot(frequency, direct_spectrum, label="Direct rational fit", linestyle=":", linewidth=2)
155 ax.set_xlabel("Frequency (cycles per observation)")
156 ax.set_ylabel("Spectral density")
157 ax.set_title("True and fitted spectral densities")
158 ax.legend()
159 fig.tight_layout()
160 fig.savefig(PNG / "problem3_spectrum_comparison.png", dpi=180)
161 plt.close(fig)
162
163 time_log_rmse = np.sqrt(np.mean((np.log(np.maximum(time_spectrum, 1e-12))
164                                - np.log(true_spectrum)) ** 2))
165 print("THETA", theta)
166 print("TIME_AR", np.round(time_ar, 5))
167 print("TIME_MA", np.round(time_ma, 5))
168 print("TIME_VARIANCE", round(time_variance, 5))
169 print("SPECTRAL_AR", np.round(spectral_ar, 5))
170 print("SPECTRAL_MA", np.round(spectral_ma, 5))
171 print("SPECTRAL_SCALE", round(spectral_scale, 5))
172 print("TIME_LOG_RMSE", round(time_log_rmse, 5))
173 print("SPECTRAL_LOG_RMSE", round(spectral_rmse, 5))

```